

Project Report

1. Cover Page

Project Title: Real-Time Lane Detection using CV

Domain: Computer Vision

Submitted by: Rishi Raj

Registration Number: 24BAI10593

Course / Subject: Computer Vision

Submitted to: VITyarthi

Semester : Fall Semester 2026-27

Date of Submission: 18/09/2026

2. Introduction

Computer Vision enables computers to obtain useful information from images and videos. Lane detection is a fundamental Computer Vision task in which road-lane boundaries are identified from a camera view. It is relevant to driver-assistance systems, road monitoring, and autonomous-vehicle research.

This project presents a real-time lane-detection system developed in Python using OpenCV. The system receives a pre-recorded road video, processes each frame, detects likely lane markings, and overlays the detected left and right lane lines on the original frame. The project deliberately uses classical image-processing methods instead of machine learning so that every stage is visible, explainable, and suitable for a second-year academic project.

The pipeline consists of grayscale conversion, Gaussian blurring, Canny edge detection, Region of Interest masking, and the Probabilistic Hough Line Transform. Detected segments are classified by slope into left and right lane candidates. Their average is calculated and smoothed across frames to reduce visual fluctuation.

3. Problem Statement

Road videos contain a large amount of visual information, while only a limited part of the image is relevant to lane boundaries. Manual identification of lane markings from each frame is impractical. Therefore, a method is required to automatically identify probable lane boundaries from road-video frames and present the result clearly.

The problem addressed in this project is to design and implement a lightweight lane-detection application that uses only classical Computer Vision techniques. The system must read a video file, identify visible left and right road lanes, display the annotated result, show FPS, and save the processed video.

4. Objectives

1. To develop a Python application for lane detection using OpenCV.
2. To read and process a road video frame by frame.
3. To extract lane-related edges using grayscale conversion, Gaussian blur, and Canny edge detection.
4. To reduce irrelevant information by applying a trapezium Region of Interest mask.
5. To identify line segments using the Hough Line Transform.
6. To separate left and right lanes using line slope and draw them in different colours.
7. To stabilize output using line averaging and temporal smoothing.

8. To display FPS and save the processed video for evaluation.

5. Functional Requirements

Module 1: Video Input and Output Management

This module opens `data/input/test_video.mp4` using OpenCV's `VideoCapture` class. It checks that the file exists and can be opened. It reads each frame, obtains video dimensions and frame rate, creates an output writer, displays the output window, and saves processed frames as `data/output/output_video.mp4`. It releases resources safely when processing completes or the user presses `Q` or `Esc`.

Module 2: Image Preprocessing and Region Selection

This module prepares each frame for line detection. It converts the colour frame into grayscale because intensity information is sufficient for basic edge detection. Gaussian blur reduces small noise and produces cleaner edges. Canny edge detection identifies strong intensity transitions. A trapezium Region of Interest mask retains the lower central road area and removes most irrelevant content such as sky, buildings, and roadside objects.

Module 3: Lane Extraction and Visualisation

This module applies the Probabilistic Hough Line Transform to ROI-masked edges. It obtains line segments, ignores unsuitable slopes, and separates the remaining segments into left and right groups. The segments in each group are averaged to form long representative lane lines. Exponential smoothing uses results from earlier frames to reduce flickering. Finally, the left lane is drawn in blue, the right lane is drawn in red, and both lines are blended with the original frame. FPS text is also displayed.

6. Non-Functional Requirements

1. **Usability:** The system should be simple to run with one command and display clear errors when the input video is missing.
2. **Performance:** The system should process frames efficiently enough to provide a live visual result for a moderate-resolution video on a normal computer.
3. **Reliability:** The application should release video resources safely and should not crash when a frame has no detectable lane line.
4. **Maintainability:** Processing logic should be organised into reusable functions in `utils.py`, while application flow remains in `main.py`.
5. **Portability:** The project should run on systems where Python, OpenCV, and NumPy are installed.
6. **Clarity:** Variables, comments, and function names should be understandable during code review and viva.

7. System Architecture

The system follows a simple input-process-output architecture. The input layer contains the road video file. The processing layer contains `main.py` and `utils.py`. `main.py` controls the video loop, line smoothing, FPS calculation, display, and saving. `utils.py` provides reusable functions for preprocessing, edge detection, ROI selection, Hough line detection, line averaging, and drawing. OpenCV performs image/video operations and NumPy supports numeric-array calculations. The output layer provides a live OpenCV window and the saved annotated video.

The complete system architecture diagram and Mermaid source are available in `docs/DIAGRAMS.md`.

8. Workflow / Process Flow

1. The user places the input road video in `data/input/test_video.mp4`.
2. The application validates and opens the video file.

3. One frame is read from the input video.
4. The frame is converted to grayscale and blurred using a Gaussian filter.
5. Canny edge detection produces an edge image.
6. A trapezium ROI mask removes edges outside the likely road area.
7. The Hough Line Transform detects candidate line segments.
8. Candidate segments are separated into left and right groups based on slope.
9. Each group is averaged; the result is smoothed using the previous frame.
10. The lane lines and FPS text are overlaid on the original frame.
11. The annotated frame is displayed and written to the output video.
12. The process repeats until the video ends or the user exits.

9. UML Diagrams

9.1 Use Case Diagram

The use case diagram identifies the user as the main actor. The user provides the input video and starts the system. The system processes the video, displays lane detection, saves the output, and accepts a stop command from the user. This diagram explains the high-level interaction between the user and the application.

9.2 Sequence Diagram

The sequence diagram describes the time order of events. The user starts `main.py`, which opens the video through OpenCV. For each frame, `main.py` calls utility functions for lane detection, receives the lane information, overlays it, writes it to the output video, and displays it. After the loop, capture and writer objects are released.

9.3 Component Diagram

The component diagram divides the project into the input video component, `main.py`, `utils.py`, external OpenCV and NumPy libraries, display output, and saved output video. It demonstrates separation of responsibilities and makes the program structure easier to understand.

Use case, sequence, component, workflow, and system architecture diagrams are provided with Mermaid code in `docs/DIAGRAMS.md`.

10. Design Decisions and Rationale

The project uses classical methods because they are computationally lightweight, transparent, and easy to explain.

Grayscale conversion reduces processing complexity because colour is not essential for basic edge detection. Gaussian blur is applied before Canny detection to reduce noise-induced edges. Canny edge detection was chosen because it produces thin and clear edge boundaries.

A trapezium ROI is used because lanes generally appear in the lower central portion of a forward-facing road video. The Hough Line Transform is appropriate because lane markings commonly form long straight or nearly straight line segments. Slope distinguishes left and right lanes: negative slopes normally correspond to the left side and positive slopes to the right side. Averaging and exponential smoothing reduce fragmented Hough segments and improve temporal stability. The project maintains modular functions to support readability and future modification.

11. Implementation Details

The project is implemented using Python, OpenCV, and NumPy. `main.py` defines project-relative paths, validates the input video, opens the capture object, creates a video writer, processes frames in a loop, calculates FPS, and manages the display window. It imports reusable functions from `utils.py`.

In `utils.py`, `preprocess()` converts BGR frames to grayscale and applies Gaussian blur. `detect_edges()` applies Canny detection. `region_of_interest()` creates a trapezium mask based on frame dimensions. `detect_lines()` runs `cv2.HoughLinesP()` to find segments. `average_slope_intercept()` removes unsuitable segments, separates them according to slope, computes representative lines, and returns them. `draw_lines()` draws thick coloured lines on an overlay before blending it with the original frame.

The output file is encoded using the `mp4v` codec and saved in the output folder. The output window can be closed safely with `Q` or `Esc`.

12. Results and Output Description

The system produces a processed video in which likely left and right lane boundaries are drawn over the original road scene. The left lane is shown in blue and the right lane in red. A green FPS label indicates approximate processing speed. The result is displayed while the program runs and saved as `data/output/output_video.mp4`.

For videos with clear, straight, and well-lit road markings, the system provides understandable lane overlays. The averaging and smoothing stages make the lines more stable than drawing individual Hough segments directly. The result demonstrates that a basic lane-detection task can be achieved without deep learning.

13. Testing Approach

The application is tested using a valid input road video and by observing the live output and saved result. Functional tests check file opening, frame processing, lane drawing, output-video creation, keyboard exit, and handling of a missing file. Visual tests verify whether colours, line positions, and FPS text are present. A detailed test-case table with input, expected output, and observed output is available in `docs/TESTING_AND_SUBMISSION.md`.

14. Challenges Faced

1. Road scenes contain many non-lane edges, including shadows, vehicles, road texture, and roadside objects.
2. Hough Transform often returns multiple short segments rather than two complete lane lines.
3. Frame-to-frame line positions can fluctuate, causing visual flicker.
4. Lane visibility changes due to faded paint, lighting variation, curves, or occlusion.
5. Video codecs and local OpenCV installation can affect successful output-video writing.

These challenges are addressed through Gaussian blur, ROI masking, slope filtering, line averaging, temporal smoothing, and explicit error checks.

15. Learnings

This project improved understanding of the practical image-processing pipeline used in Computer Vision. Important learnings include how images are represented as arrays, why preprocessing is necessary, how Canny edge detection identifies intensity changes, how ROI masking focuses a problem, and how Hough Transform converts edge points into line candidates. The project also demonstrated the importance of modular programming, error handling, video-resource management, and testing with real input data.

16. Future Enhancements

1. Add support for curved lanes using polynomial fitting.
2. Apply colour thresholding to identify white and yellow lane markings.
3. Use perspective transformation to create a bird's-eye road view.
4. Add camera calibration to correct lens distortion.
5. Add adaptive thresholds for night, rain, and shadow conditions.
6. Add lane-departure warnings.
7. Add a graphical interface for selecting input video and ROI settings.
8. Compare classical methods with a deep-learning model in a future advanced project.

17. References

1. OpenCV Documentation. *OpenCV-Python Tutorials*. Available at: <https://docs.opencv.org/>
2. Canny, J. (1986). *A Computational Approach to Edge Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence.
3. Duda, R. O., and Hart, P. E. (1972). *Use of the Hough Transformation to Detect Lines and Curves in Pictures*. Communications of the ACM.
4. Bradski, G., and Kaehler, A. (2008). *Learning OpenCV*. O'Reilly Media.
5. NumPy Documentation. Available at: <https://numpy.org/doc/>